Kế hoạch phát triển Web — phân công 2 người

Oct 30, 2026 · @Doãn Khang

Tổng quan

Phần web gồm 3 khối: **Database** lưu bài – công ty – nhãn, **Backend** (FastAPI) trả số liệu, **Frontend** (HTML, CSS, JavaScript thuần) cho người dùng chọn mã + khoảng ngày. Đề xuất chia: **Người A = Database + Backend**, **Người B = Frontend + đóng gói/deploy**. Hai người nối với nhau qua một API contract chốt trong tuần 1.

MVP (theo kế hoạch FinNews Tracker): gõ một mã (VD HPG), chọn khoảng ngày → thấy số bài tích cực / tiêu cực / trung lập, biểu đồ theo thời gian và danh sách bài có link.

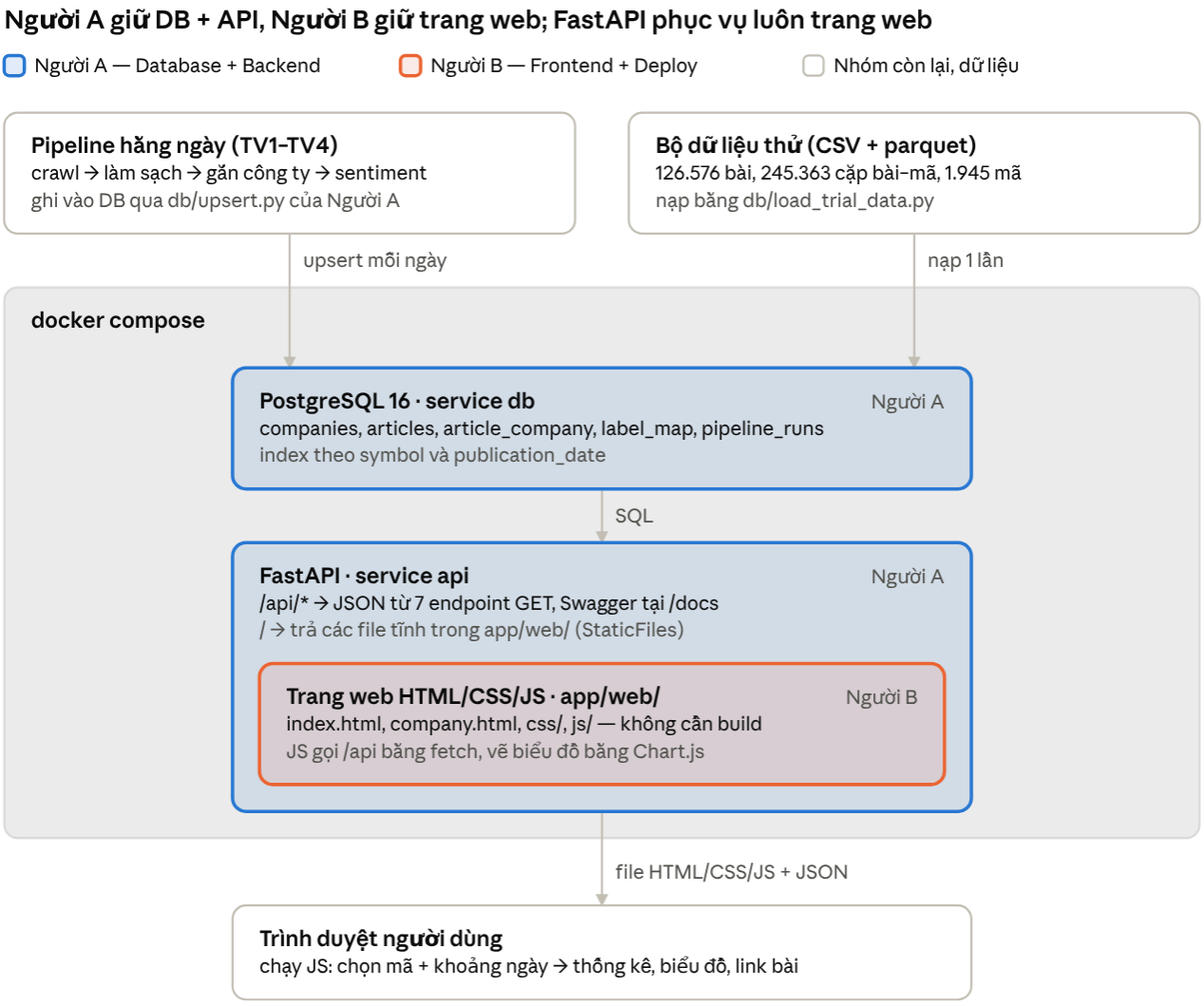
Phạm vi của 2 người: phần DB của bước ⑤ (TV2) và toàn bộ bước ⑥ (TV5). Crawl, làm sạch, gắn công ty, model do người khác làm; họ ghi vào DB qua hàm upsert do Người A cung cấp.

Dữ liệu dùng được ngay để phát triển (bộ thử, sẽ thay bằng dữ liệu crawl mới):

File	Nội dung	Dùng cho bảng
`data_news/vn_stock_symbols.csv`	1.945 mã: symbol, organ_name, old_names, brand, exchange, status	`companies`
`data_news/firm_extract/equity_news_articles.csv`	126.576 bài (110.903 là equity news), article_type, subject_symbols, link	`articles`
`data_news/equity_news_clean_content.parquet`	description, content của từng bài	`articles`
`data_news/firm_extract/firm_symbols.csv`	245.363 cặp bài–mã: position, mentions, matched_by	`article_company`
`article_scores_phobert.parquet` (output `score_corpus_phobert.py`)	prob_negative/neutral/positive theo **bài**, khoá theo link — chưa có trên máy này	nhãn baseline

Link bài là duy nhất và không rỗng trong cả 126.576 bài, nên dùng link làm khoá để sinh `article_id` được.

Kiến trúc và công nghệ



kiến trúc web · 2 service, 2 người

Dữ liệu đi một chiều: pipeline và bộ thử ghi vào PostgreSQL, API chỉ đọc, web chỉ gọi API. Công nghệ: PostgreSQL 16, FastAPI + SQLAlchemy 2, HTML + CSS + JavaScript thuần (Chart.js vẽ biểu đồ), Docker Compose. Frontend không cần Node, npm hay bước build: FastAPI trả thẳng các file trong `app/web/`, nên web và API chung một địa chỉ (`localhost:8000`) và compose chỉ có 2 service. Docker dùng để chạy PostgreSQL khi dev và đóng gói bản demo; không có Docker thì cài PostgreSQL trực tiếp cũng được.

Database (Người A)

Dùng **PostgreSQL 16 chạy bằng Docker**, 5 bảng theo schema mục 4 của kế hoạch, thêm 2 cột `is_subject`, `position` và bảng `pipeline_runs`. Mọi truy vấn của web đều lọc theo `symbol` + khoảng ngày, nên index theo hai cột đó.

Bảng	Cột	Khoá / index	Ghi chú
`companies`	symbol, organ_name, old_names, brand, exchange, status, search_text	PK symbol; GIN trigram trên search_text	search_text = mã + tên + tên cũ + brand, viết thường, bỏ dấu → gõ "hoa phat" vẫn ra HPG
`articles`	article_id, link, domain, category, title, description, content, publication_date (TIMESTAMPTZ), article_type, is_equity, created_at	PK article_id; UNIQUE link; index publication_date	article_id = 16 ký tự đầu của sha1(link)
`article_company`	article_id, symbol, mention_count, in_title, position, is_subject, context, sentiment_label, prob_pos, prob_neu, prob_neg, model_version, scored_at	PK (article_id, symbol); index symbol; FK sang 2 bảng trên	sentiment_label ∈ positive / neutral / negative (giống VALID_LABELS), NULL = chưa chấm
`label_map`	source_row_id, article_id, match_method	PK source_row_id	TV2 ghi, TV4 đọc; web không dùng
`pipeline_runs`	run_id, started_at, finished_at, n_new_articles, status	PK run_id	web hiện "dữ liệu cập nhật đến …"

Quy ước dữ liệu cần chốt:

- `is_subject = true` khi mã nằm trong `subject_symbols` (bài "tin công ty"). Chỉ cặp này được chấm sentiment và tính vào thống kê mặc định; cặp "nhắc qua" vẫn lưu.
- `in_title = (position = 'tiêu đề')`, `mention_count = mentions` lấy từ `firm_symbols.csv`.
- Nhãn baseline: nhãn cấp bài của E3 gán cho mọi công ty chủ thể, `model_version = 'E3-article-baseline'`. Khi có model theo công ty thì ghi đè và đổi model_version.

- Chưa có file điểm sentiment: loader để NULL; thêm cờ `--mock-sentiment` sinh nhãn giả (`model_version = 'mock'`) để Người B có dữ liệu vẽ, không bao giờ dùng khi demo.
- Gộp mọi file theo **link**, không theo số thứ tự `row` (row đổi khi crawl lại).

Truy vấn chính (lấy `< to + 1 ngày` vì publication_date có giờ, VD 2010-01-01 19:23):

```
SELECT ac.sentiment_label, COUNT(*) AS n
FROM article_company ac
JOIN articles a USING (article_id)
WHERE ac.symbol = :symbol
  AND ac.is_subject
  AND a.publication_date >= :date_from
  AND a.publication_date <  :date_to + INTERVAL '1 day'
GROUP BY ac.sentiment_label;
```

Việc cần làm:

- [ ] `docker-compose.yml` có service `postgres:16` + volume; `.env.example` chứa `DATABASE_URL`
- [ ] `db/schema.sql` tạo 5 bảng, FK, index, `CREATE EXTENSION pg_trgm`
- [ ] `db/load_trial_data.py`: nạp theo thứ tự companies → articles → article_company → nhãn baseline; chạy lại không trùng (`INSERT … ON CONFLICT DO UPDATE`)
- [ ] `db/upsert.py`: `upsert_articles(df)`, `upsert_article_company(df)`, `update_sentiment(df, model_version)` cho job hằng ngày của TV2 gọi
- [ ] Kiểm tra số dòng khớp file nguồn: 1.945 công ty, 126.576 bài, 245.363 cặp bài–mã
- [ ] `EXPLAIN ANALYZE` truy vấn chính với HPG cả 2010–2023; mục tiêu dưới 200 ms
- [ ] Script `pg_dump` / restore để chia sẻ DB qua nơi lưu chung của nhóm

Backend (Người A)

Dùng **FastAPI + SQLAlchemy 2 (Core, viết SQL thẳng)** với 7 endpoint chỉ đọc. Trang Swagger tự sinh tại `/docs` chính là hợp đồng với Người B, nên có bản chạy được (trả dữ liệu giả) ngay tuần 1.

Cấu trúc thư mục đề xuất:

```
app/backend/
  main.py              # tạo app, gắn router /api, mount app/web/ tại /
  config.py            # đọc DATABASE_URL từ biến môi trường
  database.py          # engine + get_conn()
  schemas.py           # Pydantic model của mọi response
  queries.py           # toàn bộ câu SQL
  routers/companies.py
  routers/meta.py
  tests/               # pytest + TestClient
  requirements.txt     # riêng cho web: fastapi, uvicorn, sqlalchemy,
psycopg, pytest, httpx
  Dockerfile           # python:3.12-slim, copy cả app/web/
```

`requirements.txt` riêng để image web không phải cài torch/transformers.

Endpoint	Tham số	Trả về
`GET /api/health`	—	`{status: "ok"}`
`GET /api/meta`	—	min_date, max_date, last_updated, model_version, danh sách nguồn
`GET /api/companies`	`q`, `limit=10`	`[{symbol, organ_name, exchange, status}]`; mã khớp đúng xếp đầu, rồi theo độ giống (pg_trgm)
`GET /api/companies/{symbol}`	—	thông tin 1 công ty; 404 nếu không có
`GET /api/companies/{symbol}/summary`	`from`, `to`, `include_mentions=false`	`{positive, neutral, negative, unscored, total, avg_score}`
`GET /api/companies/{symbol}/timeseries`	`from`, `to`, `bucket=auto|day|week|month`	`[{period, positive, neutral, negative}]`, kể cả kỳ bằng 0

Endpoint	Tham số	Trả về
`GET /api/companies/{symbol}/articles`	`from`, `to`, `label`, `page=1`, `page_size=20` (tối đa 100), `sort=date_desc|score_desc|score_asc`	`{items: [{article_id, title, link, domain, publication_date, label, score}], total, page}`

Quy tắc xử lý:

- `score = prob_pos − prob_neg`, cùng công thức với `net_sentiment_score` đang dùng.
- `bucket=auto`: khoảng ⩽ 60 ngày → ngày, ⩽ 2 năm → tuần, dài hơn → tháng (`date_trunc`). Kỳ không có bài vẫn trả 0 (`generate_series`) để biểu đồ không bị nhảy.
- Thiếu `from`/`to` → mặc định 90 ngày cuối tính tới `max_date` (bộ thử dừng ở 12/2023, lấy theo hôm nay sẽ ra rỗng). `from > to` → 422.
- Viết hoa `symbol` trước khi truy vấn. Tìm kiếm bỏ dấu dùng chung một hàm với loader (nhớ đ → d), đặt ở một module chung.
- Danh sách bài không trả `content` (nặng). Cần xem chi tiết thì thêm `GET /api/articles/{article_id}` trả câu ngữ cảnh — làm sau MVP.
- Web và API cùng cổng 8000 nên không cần CORS. Mount `StaticFiles(directory="app/web", html=True)` tại `/` ở **cuối** `main.py`, sau các router `/api`, để không che mất API.
- Test chạy trên Postgres thật (DB riêng `finnews_test`), vì SQL dùng `date_trunc`, `pg_trgm` mà SQLite không có.

Việc cần làm:

- [ ] Khung FastAPI + `/api/health` + mount `app/web/` tại `/`; chạy được `uvicorn app.backend.main:app --reload`
- [ ] `schemas.py` + mọi endpoint trả dữ liệu giả cố định → gửi link `/docs` cho Người B trong tuần 1
- [ ] Thay dữ liệu giả bằng SQL thật theo thứ tự: companies → summary → articles → timeseries → meta
- [ ] Validate tham số, khoảng mặc định, giới hạn phân trang
- [ ] pytest: mã không tồn tại → 404; khoảng không có bài → toàn 0; total của summary = tổng timeseries = total của articles
- [ ] Dockerfile (`python:3.12-slim` + uvicorn), copy cả `app/web/` để một image chạy cả API lẫn web

- [ ] Thêm mục “Chạy web” vào README

Frontend (Người B)

Dùng **HTML + CSS + JavaScript thuần**, không framework, không bước build: các file trong `app/web/` được FastAPI trả thẳng tại `http://localhost:8000/`, cùng địa chỉ với API. Có 2 trang; mọi lựa chọn (mã, ngày, nhãn, trang) nằm trên URL để gửi link là mở đúng màn hình. Trong lúc Người A làm backend, Người B làm trên các file JSON mẫu.

Công cụ:

- **`fetch`** gọi API; **`<script type="module">`** để chia JS thành nhiều file (`import` / `export`), trình duyệt nào cũng chạy được
- **Chart.js 4** — biểu đồ cột chồng, tooltip, bắt sự kiện click cột; tải file `chart.umd.min.js` về `app/web/vendor/` để demo không cần mạng
- **`<input type="date">`** có sẵn của trình duyệt để chọn ngày; **`Intl.DateTimeFormat('vi-VN')`** định dạng dd/MM/yyyy, không cần thư viện ngày
- **CSS tự viết** — biến màu trong `:root`, Flexbox / Grid, `@media` cho điện thoại, `prefers-color-scheme` cho chế độ tối
- **JSON mẫu** trong `app/web/mock/`, chép từ ví dụ ở `/docs`; đặt `USE_MOCK = true` trong `api.js` thì web đọc file mẫu thay vì gọi API

Trang	Gồm	Chi tiết
`index.html`	ô tìm kiếm, vài mã gợi ý, dòng “dữ liệu cập nhật đến …”	gõ xong 300 ms mới gọi `/api/companies`; Enter hoặc click → `company.html?symbol=HPG`
`company.html?symbol&from&to&label&sort&page`	đầu trang, chọn ngày, thẻ số, biểu đồ, bảng bài	xem danh sách bên dưới

Trang công ty, từ trên xuống. Mỗi phần là một hàm `render…` nhận dữ liệu và vẽ vào một `<section>` có sẵn trong `company.html`:

1. **renderHeader** — mã, tên công ty, sàn; nhãn “đã huỷ niêm yết” khi status khác listed.
2. **renderDateRange** — nút 1 tháng / 3 tháng / 1 năm / tất cả + 2 ô `<input type="date">`; đặt `min` / `max` theo min_date–max_date lấy từ `/api/meta`.
3. **renderCards** — Tích cực, Tiêu cực, Trung lập, Tổng, kèm % trên tổng; click thẻ → lọc bảng theo nhãn đó.

4. **renderChart** — Chart.js cột chồng 3 màu theo kỳ, tooltip số bài; click một cột → đặt from/to về kỳ đó. Dưới biểu đồ ghi "Nhãn do mô hình tự động chấm · model_version".
5. **renderTable** — tiêu đề (link mở tab mới, `rel="noopener"`), nguồn, ngày dd/MM/yyyy, nhãn, điểm; lọc nhãn, sắp xếp, 20 bài/trang.

Luồng chạy của `company.js`: đọc URL thành một object `state` → gọi `/summary`, `/timeseries`, `/articles` song song bằng `Promise.all` → gọi các hàm `render…`. Người dùng đổi bộ lọc → `setState(patch)`: sửa `state`, ghi lên URL bằng `history.pushState`, tải lại dữ liệu. Bấm Back → sự kiện `popstate` → đọc lại URL và vẽ lại.

Quy ước giao diện:

- Một bộ màu cho 3 nhãn, đặt thành biến CSS (`--positive`, `--negative`, `--neutral`) và dùng chung ở thẻ, biểu đồ, nhãn: xanh lá = tích cực, đỏ = tiêu cực, xám = trung lập. Chart.js lấy màu từ biến CSS bằng `getComputedStyle`.
- Đủ 4 trạng thái: đang tải (khung xám nhấp nháy), không có bài trong khoảng này, lỗi mạng (nút thử lại), mã không tồn tại.
- Màn hình ⩽ 768 px: thẻ xếp 2×2, bảng chuyển thành danh sách thẻ.

Lưu ý khi viết JS thuần:

- Chữ lấy từ API (tiêu đề bài, tên công ty) gán bằng `textContent`, không ghép vào `innerHTML`: tiêu đề báo có thể chứa `<`, `&` hoặc mã độc.
- Đổi bộ lọc liên tục thì kết quả cũ có thể về sau kết quả mới: huỷ request cũ bằng `AbortController` trước khi gọi request mới.
- Vẽ lại biểu đồ: gọi `chart.destroy()` trước khi tạo biểu đồ mới (hoặc sửa `chart.data` rồi `chart.update()`), nếu không các biểu đồ sẽ chồng lên nhau.
- Module JS không chạy khi mở file trực tiếp (`file://`): luôn mở qua `http://localhost:8000` (uvicorn), hoặc chạy `python -m http.server` trong `app/web/` khi chỉ dùng JSON mẫu.

Cấu trúc thư mục:

```
app/web/
  index.html          # trang chủ
  company.html        # trang công ty, các <section> rỗng để JS vẽ vào
  css/style.css       # biến màu, layout, @media điện thoại, chế độ tối
  js/api.js           # getJson + 6 hàm gọi API, báo lỗi; cờ USE_MOCK
  js/format.js        # tên nhãn tiếng Việt, màu, định dạng ngày và %
  js/search.js        # ô tìm kiếm dùng chung cho 2 trang
  js/home.js          # trang chủ
  js/company.js       # state <-> URL, gọi API, các hàm render…
  vendor/chart.umd.min.js
  mock/               # JSON mẫu: meta, companies, summary, timeseries,
articles
```

Việc cần làm:

- [ ] Khung `app/web/` : 2 file HTML, `style.css` , tải Chart.js về `vendor/`
- [ ] `api.js` + JSON mẫu theo `/docs` ; cờ `USE_MOCK` để chuyển giữa JSON mẫu và API thật
- [ ] Trang chủ với ô tìm kiếm (chờ 300 ms, phím lên/xuống, Enter)
- [ ] Trang công ty: lần lượt đầu trang → chọn ngày → thẻ → bảng → biểu đồ
- [ ] Đồng bộ `state` với URL; reload hoặc bấm Back vẫn đúng màn hình
- [ ] 4 trạng thái loading / rỗng / lỗi / 404; responsive; chế độ tối
- [ ] Tắt `USE_MOCK` , nối API thật; so số trên web với truy vấn SQL cho 3 mã
- [ ] Kiểm tra `docker compose up` trên máy sạch mở được web ở `http://localhost:8000`
- [ ] Kịch bản demo: 2–3 mã nhiều bài, chỉ ra một bài nhiều công ty có nhãn khác nhau theo từng công ty

Phân công 2 người

Người A giữ dữ liệu (DB + API), Người B giữ phần người dùng nhìn thấy (giao diện + deploy). Hai người chỉ phụ thuộc nhau ở API contract, nên làm song song từ ngày đầu.

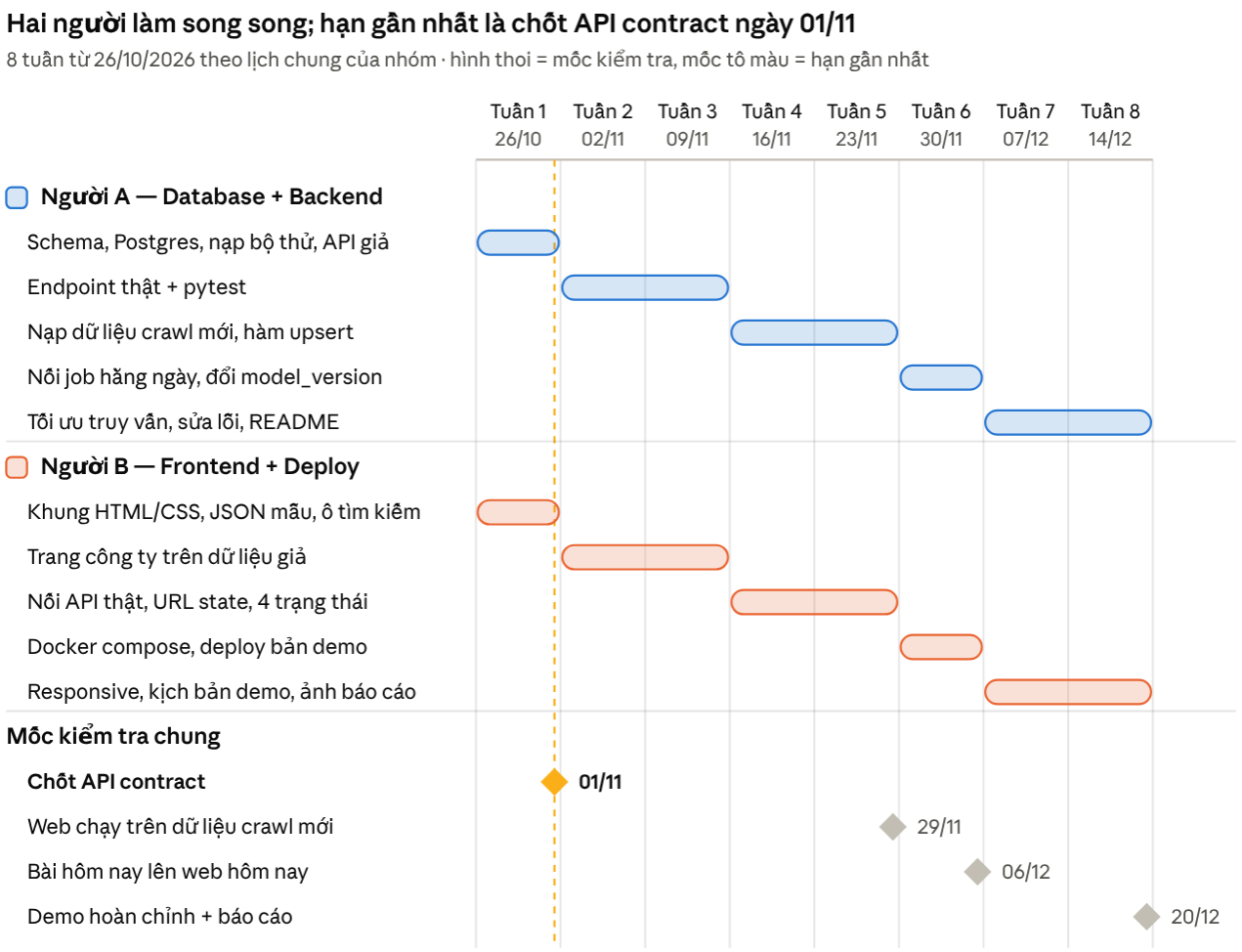
	Người A — Database + Backend	Người B — Frontend + Deploy
Thư mục sở hữu	`db/` , `app/backend/` , service `db` + `api` trong compose	`app/web/` (2 trang HTML, CSS, JS, JSON mẫu)
Kỹ năng chính	SQL, Python, FastAPI	HTML, CSS, JavaScript (fetch, DOM), Chart.js
Nhận từ	TV1–TV4: bài crawl mới, kết quả gắn công ty, điểm sentiment	Người A: API + trang `/docs`

	Người A — Database + Backend	Người B — Frontend + Deploy
Giao cho	Người B: API chạy được; TV2: các hàm upsert cho job hằng ngày	Cả nhóm: link demo + kịch bản demo
Đầu ra cuối	DB có dữ liệu mới, 7 endpoint có test	Web chạy bằng `docker compose up`

Cách phối hợp:

- Tuần 1, A gửi contract (`schemas.py` + `/docs` trả dữ liệu giả). B không chờ backend thật, làm trên JSON mẫu chép từ ví dụ ở `/docs` .
- A đổi response → sửa `schemas.py` , ghi trong mô tả PR, báo B; B sửa `api.js` và JSON mẫu theo.
- Tuần 4–5 hai người cùng so số trên web với SQL. Từ tuần 6 ai rảnh hơn thì hỗ trợ: B viết thêm test API, A kiểm tra số liệu trên UI.

Lộ trình 8 tuần



lộ trình 8 tuần · 2 người, 4 mốc

Mỗi người có 5 chặng nối tiếp. Mốc tuần 5 và tuần 6 lấy từ lịch chung của nhóm, nên phụ thuộc tiến độ crawl (TV1) và job hằng ngày (TV2); nếu dữ liệu mới chậm, web vẫn demo được trên bộ thử.

Checklist nghiệm thu MVP web:

- [ ] Gõ “hoa phat” hoặc “HPG” đều ra HPG trong ô tìm kiếm
- [ ] Chọn HPG + một khoảng ngày → thấy thẻ số, biểu đồ và bảng bài có link mở đúng bài gốc
- [ ] Tổng các thẻ = tổng các cột biểu đồ = số bài trong bảng (cùng khoảng ngày)
- [ ] Số trên web khớp truy vấn SQL kiểm tra tay cho 3 mã
- [ ] Mở lại link chia sẻ ra đúng mã, khoảng ngày, bộ lọc
- [ ] `docker compose up` trên máy sạch chạy được cả 2 service và mở được web ở `http://localhost:8000`
- [ ] Bài crawl hôm nay xuất hiện trên web trong ngày, sau khi job hằng ngày chạy
- [ ] Dùng được trên điện thoại

Quy ước làm việc chung

- **Git**: nhánh `feat/db-…`, `feat/api-…`, `feat/web-…`; mở PR vào `main`, người kia review trong 24 giờ. Không commit dữ liệu (`data_news/` đã ignore) và file `.env`.
- **Contract**: đổi API = sửa `schemas.py` + ghi rõ trong PR; không đổi tên field mà không báo trước.
- **Ngày giờ**: query và response dùng `YYYY-MM-DD`; publication_date lưu kèm múi giờ +07:00, API lọc theo ngày giờ Việt Nam.
- **Nhãn**: dữ liệu và API luôn dùng `positive` / `neutral` / `negative`; chỉ giao diện dịch sang tiếng Việt.
- **Họp**: 15 phút, 2 lần/tuần giữa A và B; cuối tuần demo cho cả nhóm 5 người.
- **Xong một việc** = có test hoặc ảnh chụp màn hình trong PR, README cập nhật, chạy được bằng `docker compose up`.